Game flow in detail

Space Invaders on the DE2-115: what happens from power-on to the end screen

1. The whole flow at a glance

The game is controlled by one 2-bit register, **state**, which holds one of four values. It decides what the game logic is allowed to do and which picture is drawn. The flow below shows how the game moves between those four states.

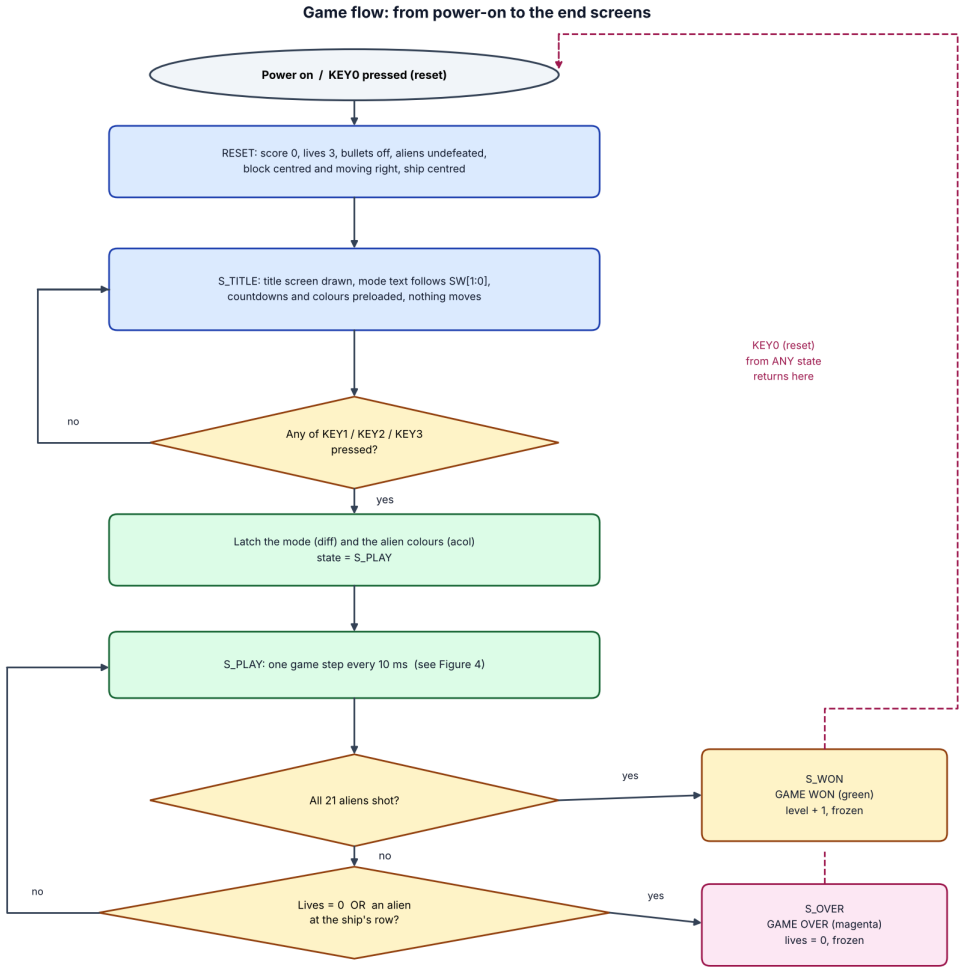


Figure 1. Overall flow. Reset (KEY0) can interrupt any state and returns to the title screen.

State	Value	Screen shown	What runs	Leaves when
S_TITLE	0	Title screen	Only the preparation: mode copied, colours and timers preloaded. Nothing moves.	any of KEY1 / KEY2 / KEY3 pressed
S_PLAY	1	The game	Everything: ship, bullets, aliens, fire, hits, score, lives.	lives reach 0, an alien reaches the ship's row, or all 21 aliens shot
S_OVER	2	GAME OVER (magenta)	Nothing: all frozen.	only KEY0 (reset)
S_WON	3	GAME WON (green)	Nothing: all frozen.	only KEY0 (reset)

2. Stage 1: power-on, reset and the title screen

Reset. When the board powers up, or whenever KEY0 is pressed (**not_reset** = 0), every game register is set to its start value: score 0, level 0, lives 3, no bullets, no alien shot, the alien block centred and set to move right, the ship in the middle (x = 300), and **state = S_TITLE**. The random-number generators (LFSRs) are the only things that are *not* reset: they keep running.

Title screen. The title shows the rainbow SPACE INVADERS, an underline, a row of aliens, “PRESS ANY KEY TO START”, a decorative ship and bullet, the striped frame and the stars. The “MODE EASY / MEDIUM / HARD” text follows the slide switches live, so you can change the mode while the title is up. Nothing in the game moves. The logic, however, is busy preparing on every clock:

- **diff <= mode**: the current switch setting is copied into the difficulty register (the last copy before the key press is the one that stays).
- **acol**: random bits are copied into the alien colour register (the colours of this game).
- **The countdown timers** (block descent, block sideways, shooters A and B) are preloaded with random starting waits.
- **In HARD mode** each alien's spawn timer is loaded with a row-dependent delay, and the aliens are marked as not on screen.

the title branch of the state machine (src/graphics.v, lines 633-654)

```
end else if (state == S_TITLE) begin
    //---------------------------------------------------------
    // TITLE: nothing about the game itself moves. Keep sampling the
    // mode switches (what they read at the instant a key is pressed
    // is what gets latched into `diff`), keep the three random
    // countdowns pre-loaded from the live mode, and wait for ANY of
    // the three game keys. On the start press, nes_a/nes_b are
    // preset to 1 so that if the key that started the game is the
    // shoot key it is NOT also seen as a fresh rising edge (i.e. no
    // free bullet).
    //---------------------------------------------------------
    diff               <= mode_n;
    acol               <= lfsr2[TOTAL_ALIENS-1:0] ^ lfsr[31 -: TOTAL_ALIENS];
    descend_counter    <= desc_delay(mode_n[0], lfsr);
    horiz_counter      <= horiz_delay(mode_n[0], lfsr);
    alien_fire_counter <= fire_delay(mode_n, lfsr);
    alien_fire_counter_b <= fire_delay(mode_n, lfsr2);
    if (any_key) begin
        state <= S_PLAY;
        nes_a <= 1'b1;
        nes_b <= 1'b1;
    end
```



Title - MODE EASY (SW1=0,SW0=0) Title - MODE MEDIUM (SW0=1) Title - MODE HARD (SW1=1)

Figure 2. The title screen in the three modes (the MODE text follows SW[1:0]).

3. Stage 2: the game starts

The moment one of KEY1 (right), KEY2 (left) or KEY3 (shoot) is pressed, **state** becomes S_PLAY. Everything copied on that last clock is frozen: from now on flipping the switches does nothing, and the colours of the aliens stay fixed for the whole game. **nes_a** and **nes_b** (the shoot-key edge detector) are set to 1, so a shoot key that was used to start the game does not also fire a bullet.

What the player sees at the start	EASY / MEDIUM	HARD
Aliens	all 21 on screen at once, as a 7 x 3 block centred on the screen, moving right	none yet; they appear one by one on the top edge, row 0 first, row 1 after about 1 s, row 2 after about 2 s
Ship	centre of the bottom row (x = 300)	same
Score and lives	SCORE 000, LIVES 3 (top right)	same

What the player sees at the start	EASY / MEDIUM	HARD
Screen	starfield and striped frame, same as the title	same

4. Stage 3: what runs while playing

Two things run at different rates. The **picture** is redrawn continuously, pixel by pixel (about 60 frames per second), always from whatever values the registers hold at that moment. The **game step** updates those registers once every 10 ms (the pulse **move_tick**, about 100 times per second). The two are independent.



Figure 3. The picture and the game step run at different rates.

In every game step the logic goes through the following decisions, in this order. They are all in one clocked block, and when two steps assign the same register in the same clock, the later one wins. That is why the checks that can end the game are at the end, and the touch check is last of all.

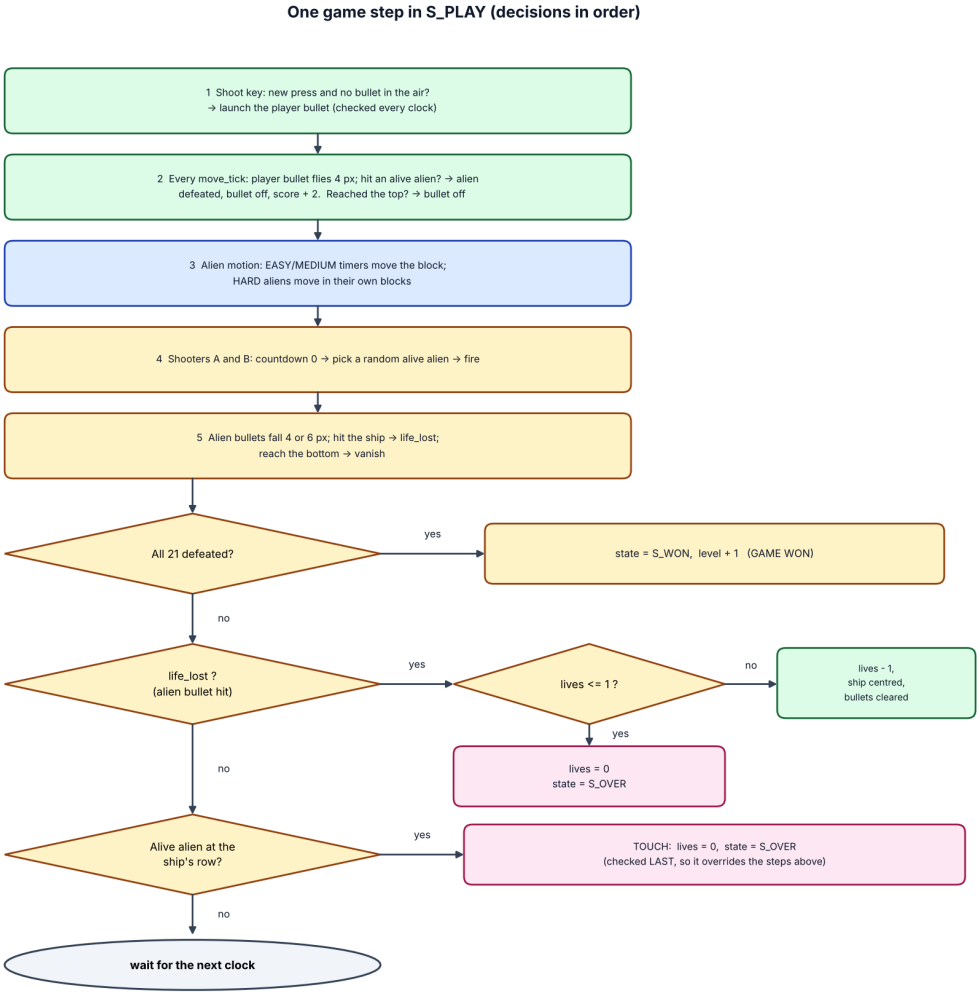


Figure 4. One game step in S_PLAY. The ship moves in its own block (every 5th step, 5 px while a key is held).

Steps 1 and 2: your bullet

A shot is launched only on a new key press while no bullet of yours is in the air. The bullet then climbs 4 px each game step (400 px per second). On every step it is tested against all alive aliens; a hit removes the alien, switches the bullet off and adds 2 to the score. A bullet that reaches the top of the alien area without a hit disappears.

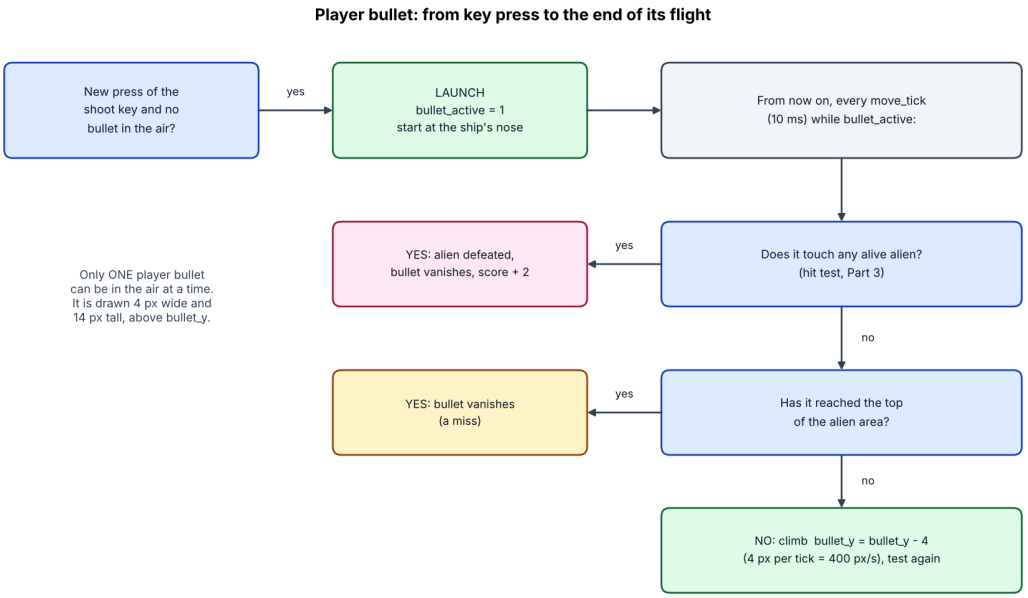


Figure 5. The life of your bullet.

Steps 3 and 4: aliens move and fire

EASY/MEDIUM: two random countdown timers move the block sideways and down. **HARD:** each alien runs its own timers in its own block, independent of the others. Two shooters, A and B, each have their own random countdown. When a countdown reaches 0, a random alien is picked; if it is alive and that shooter has no bullet in the air, it fires from the bottom centre of its box. If the pick is not alive, another pick is tried on the very next step.

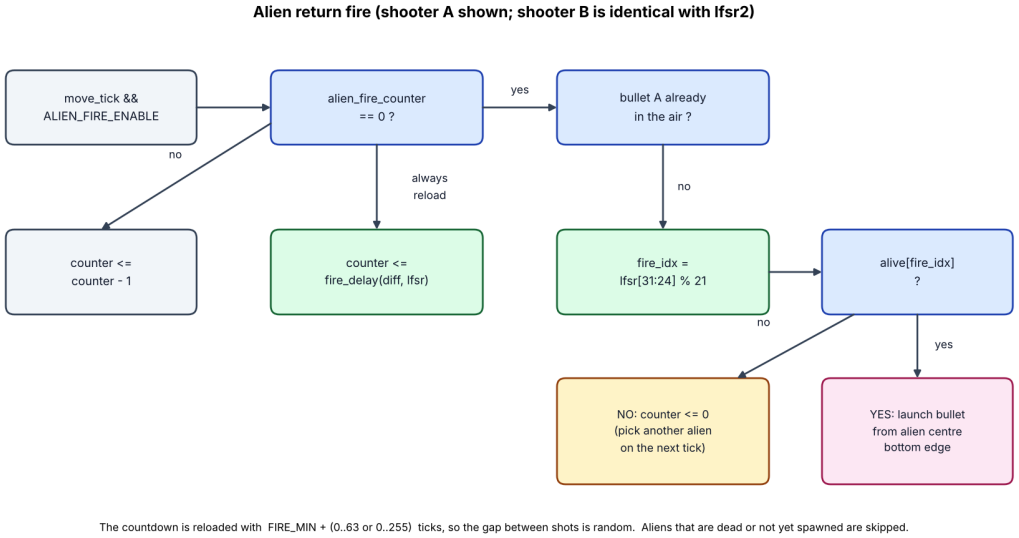


Figure 6. How an alien shot is decided (shooter A; B is identical).

Step 5: alien bullets

An alien bullet falls straight down with the x it had at launch, 4 px per step in EASY and 6 px per step in MEDIUM and HARD. If it overlaps the ship, a life is lost (**life_lost**); if it reaches the bottom frame it vanishes.

5. Stage 4: events and what they change

The three checks at the end of every step decide whether the game continues. The code is in this order; the touch check is last.

win, life lost and touch (src/graphics.v, lines 839-842, 849-860, 866-875)

```
if (!flip_counter && &defeated) begin
    level_reg <= level_reg + 2'd1;
    state     <= S_WON;
end

if (life_lost) begin
    if (lives_reg <= 2'd1) begin
        lives_reg <= 2'd0;
        state     <= S_OVER;
    end else begin
        lives_reg           <= lives_reg - 2'd1;
        life_lost_pending   <= 1'b1;
        bullet_active       <= 1'b0;
        alien_bullet_active <= 1'b0;
        alien_bullet_b_active <= 1'b0;
    end
end

touch = 1'b0;
for (k = 0; k < TOTAL_ALIENS; k = k + 1) begin
    cy = apy_flat[10*k +: 10];
    if (alive[k] && (cy + ALIEN_H >= SHIP_Y))
        touch = 1'b1;
end
if (touch) begin
    lives_reg <= 2'd0;
    state     <= S_OVER;
end
```

Event	Test	What changes	Next state
Your bullet hits an alien	box overlap, alien alive	defeated bit set, bullet off, score + 2	S_PLAY
All 21 aliens defeated	&defeated = 1	level + 1	S_WON
Alien bullet hits the ship, lives 3 or 2	box overlap	lives - 1, ship back to the centre, all bullets cleared, aliens stay where they are	S_PLAY
Alien bullet hits the ship, last life	box overlap and lives <= 1	lives = 0	S_OVER
An alive alien reaches the ship's row	y + 18 >= 440	lives = 0 (whatever it was)	S_OVER

6. Stage 5: the end screens

In S_OVER and S_WON the game block does nothing at all: the ship, bullets and aliens are frozen exactly where they were, and the picture is replaced by the message in the middle of a black screen. **game_over_o** or **game_won_o** goes high. The score and level remain on the HEX displays.

frozen in OVER / WON (src/graphics.v, lines 655-659)

```
end else if (state != S_PLAY) begin
    // OVER or WON: freeze the whole game in place. The "GAME OVER" /
    // "GAME WON" text drawn in the pixel-colour mux below is static,
    // so there is nothing left to update here. Only reset (KEY[0])
    // leaves these states -- back to the title screen.
```

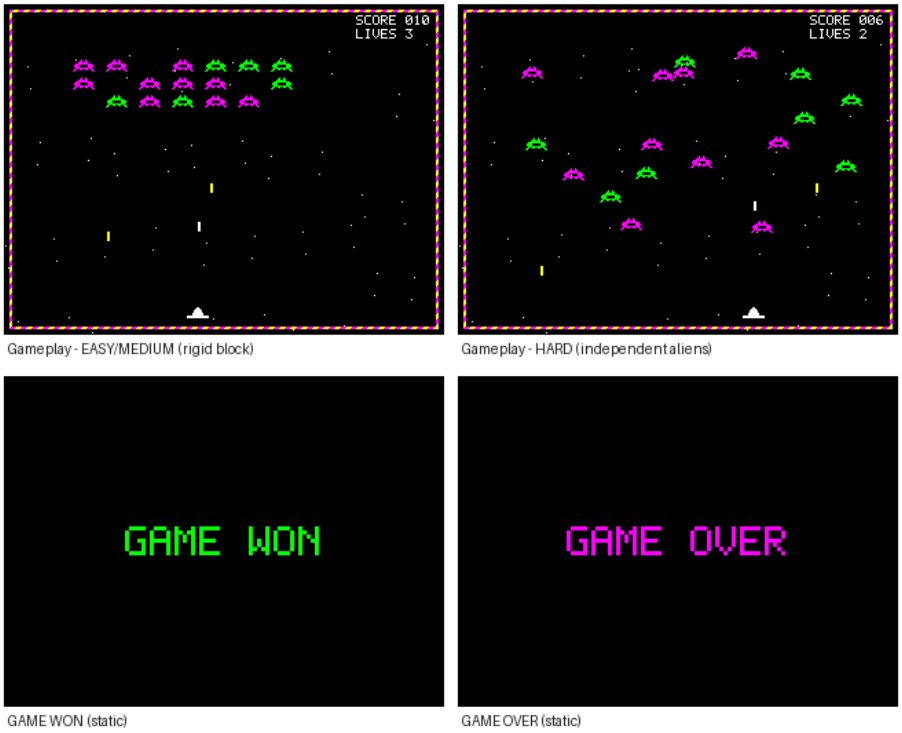


Figure 7. Gameplay (top) and the two end screens (bottom).

7. Stage 6: starting again

Only **KEY0** leaves the end screens. It resets everything (Stage 1) and the title screen returns. The mode switches can be changed, and because the random-number generators kept running the whole time, the next game starts at a different point in the random sequence: the alien movement, the firing and the colours are different every game.

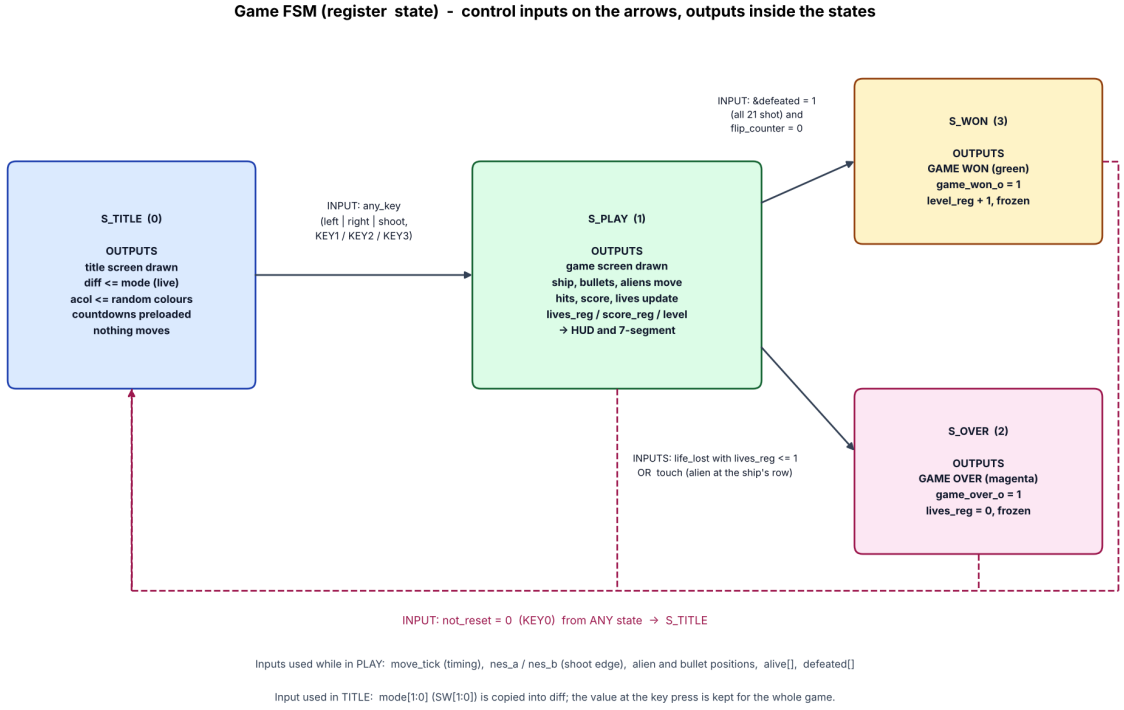


Figure 8. The game state machine with its control inputs on the arrows and its outputs inside the states.

8. A game on a time line

The picture below shows how the pieces fit together in time for one HARD game. It is a simplified illustration (not every shot is shown), meant to show the order of events: the title screen, the key press, the aliens appearing row by row, your shots and hits, alien shots, lives being lost, and finally the win.

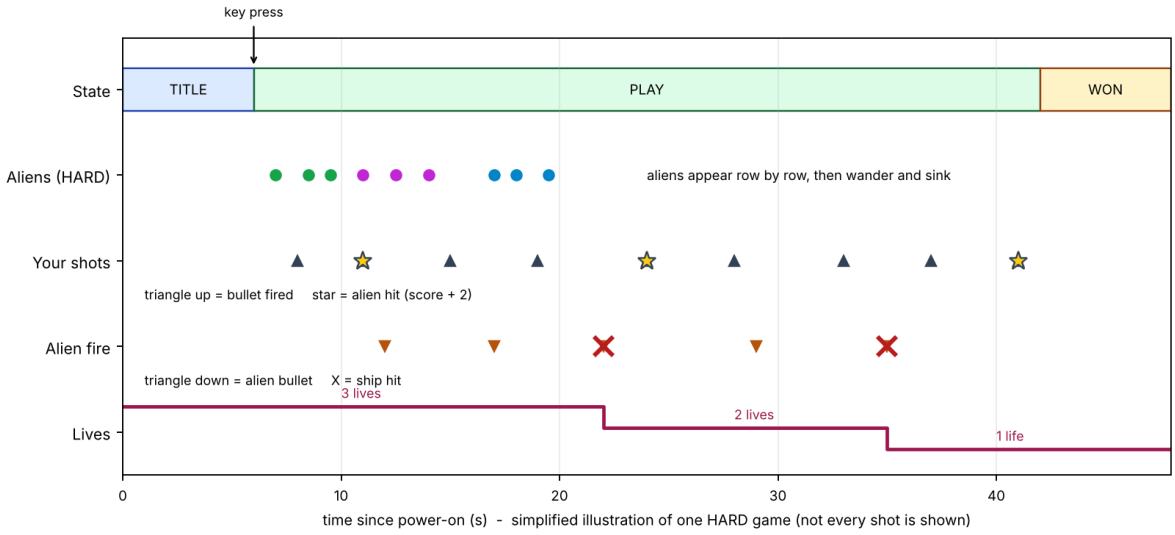


Figure 9. A simplified time line of one HARD game.

9. Quick reference

Signal	Role in the flow
not_reset (KEY0)	reset from any state, back to the title
any_key (KEY1/2/3)	starts the game from the title screen
mode / diff	switch setting on the title screen, latched into diff at the start key press
move_tick	the 10 ms pulse that triggers a game step
state	S_TITLE 0, S_PLAY 1, S_OVER 2, S_WON 3; also chooses the picture
defeated[k], &defeated	which aliens are shot; all set means the win
life_lost, lives_reg	an alien bullet hit and the lives left
touch	a live alien reached the ship's row
game_over_o, game_won_o	flags that mirror the end states

Status. This flow is taken from the Verilog as written. The design has been run on the board, but the testbenches and the Quartus timing report have not been checked yet.